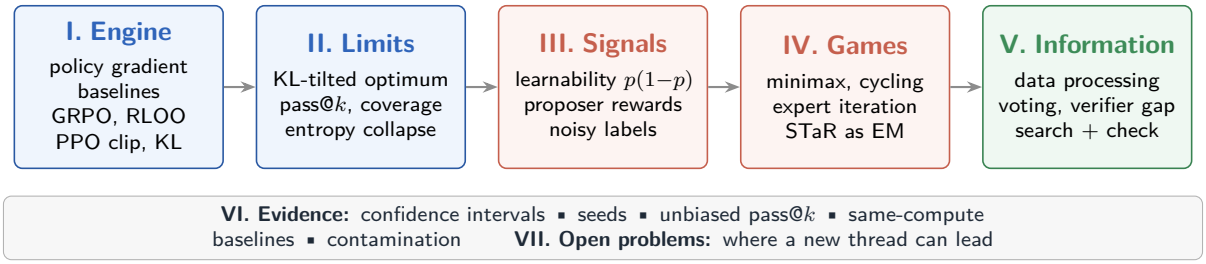
NOTES ▪ VERSION 1.3 ▪ NOVEMBER 2026

# Foundations of Self-Play for LLM Reasoning

The theory you need to study how a model can learn to reason when no one knows the answer

Chandak Chakma ▪ Personal study notes ▪ chandakchakma.github.io

**Purpose.** Challenger–Solver self-play asks one model to write its own problems, solve them, and judge its own answers. This guide collects the concepts, results and formulas behind that idea. Every topic comes with a definition, a short derivation, a worked example with real numbers, and the reason it matters for self-play. **Part V is the heart of the guide:** it asks where new knowledge can come from in a loop that has no outside teacher.




**How to read.** Parts I–II are the engine every method shares. Part III is reward design for the Challenger. Part IV explains the dynamics of two learners. Part V is the theory of self-improvement. Part VI is the evidence standard reviewers expect. Exercises close each part; answers are in Appendix B. Appendix A is a one-page formula sheet.

## Notation

| | | | |
|---|---|---|---|
| $x$ | problem (prompt) | $\pi_\theta(y \mid x)$ | Solver policy |
| $y$ | full response, ending in an answer | $\pi_{\text{ref}}$ | frozen reference (base) model |
| $z$ | reasoning trace (rationale) | $R(x, y)$ | reward, usually in $\{0, 1\}$ |
| $p(x)$ | Solver's success probability on $x$ | $G$ | samples per problem (group size) |
| $\hat{A}_i$ | advantage of sample $i$ | $\beta,\ \varepsilon$ | KL weight, clip range |
| $\hat{p}$ | agreement with the majority answer | $k$ | sample budget in pass@$k$ |

PART I
# The RL engine

## 1 LLM generation as reinforcement learning

A problem $x$ is the state and the whole response $y$ is the action; a verifier, or a pseudo-label in self-play, returns one scalar reward at the end. This *bandit view* is the one used by GRPO, R-Zero and SOAR. The policy factorizes over tokens:

$$\pi_\theta(y \mid x) = \prod_{t=1}^{|y|} \pi_\theta(y_t \mid x, y_{<t}), \qquad J(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[R(x,y)\big].$$

With a binary reward, $J(\theta) = \mathbb{E}_x[p_\theta(x)]$: training maximizes the average success probability.

## 2 The policy gradient

**Key result | log-derivative trick**

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,\, y\sim\pi_\theta}\Big[R(x,y)\, \nabla_\theta \log \pi_\theta(y \mid x)\Big], \qquad \nabla_\theta \log \pi_\theta(y \mid x) = \sum_t \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}).$$

**Derivation.** For a fixed $x$,

$$\nabla_\theta \sum_y \pi_\theta(y \mid x) R(x,y) = \sum_y \pi_\theta(y \mid x)\, R(x,y)\, \frac{\nabla_\theta \pi_\theta(y \mid x)}{\pi_\theta(y \mid x)} = \mathbb{E}_{y\sim\pi_\theta}[R\, \nabla_\theta \log \pi_\theta(y \mid x)].$$

The expectation can be estimated from samples, so we never need the sum over all responses.

**Why it matters for self-play.** Every method in this guide (PPO, GRPO, RLOO, R-Zero, Absolute Zero, SOAR) estimates this one expectation. They differ only in how they weight each sample, and in where $R$ comes from.

## 3 Baselines and advantages

**Key result | a baseline adds no bias**

For any $b(x)$ that does not depend on the sampled $y$:

$$\mathbb{E}_{y\sim\pi_\theta}\big[b(x)\, \nabla_\theta \log \pi_\theta(y \mid x)\big] = b(x) \sum_y \nabla_\theta \pi_\theta(y \mid x) = b(x)\, \nabla_\theta \sum_y \pi_\theta(y \mid x) = b(x)\, \nabla_\theta 1 = 0,$$

so $\nabla_\theta J = \mathbb{E}\big[(R - b(x))\, \nabla_\theta \log \pi_\theta\big]$. The term $\hat{A} = R - b$ is the *advantage*. A good baseline, close to $\mathbb{E}[R \mid x]$, lowers the variance a lot.

**Worked example | why the baseline matters**

Four samples get rewards $(1, 1, 1, 0)$. Without a baseline, the three correct samples are pushed up by $1$ and the wrong one gets weight $0$: it is never pushed down. With $b = 0.75$ the weights become $(+0.25, +0.25, +0.25, -0.75)$. Now the single failure carries the largest signal, which is what we want on an easy problem.

**Pitfall.** A baseline that uses the sample's *own* reward, such as the group mean including $R_i$, is no longer independent of $y_i$. The leave-one-out baseline below fixes this.

## 4 Group-relative advantages: GRPO and RLOO

**Definition | GRPO (Shao et al., 2024)**

Sample $G$ responses for one problem. The advantage of response $i$, shared by all its tokens, is

$$\hat{A}_i = \frac{R_i - \mathrm{mean}(R_1, \ldots, R_G)}{\mathrm{std}(R_1, \ldots, R_G) + \epsilon_{\mathrm{std}}}.$$

The loss combines a PPO-style clipped ratio (Section 5) with a KL penalty to $\pi_{\mathrm{ref}}$:

$$\mathcal{J}_{\mathrm{GRPO}} = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t}\min\Big(\rho_{i,t}\hat{A}_i,\ \mathrm{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big)\right] - \beta\, D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}).$$

**Definition | RLOO (Ahmadian et al., 2024)**

$$\hat{A}_i = R_i - \frac{1}{G-1}\sum_{j\neq i} R_j.$$

The baseline uses only the *other* samples, so it is independent of $y_i$ and the estimator is unbiased.

**Key result | the two are almost the same**

$R_i - \frac{1}{G-1}\sum_{j\neq i} R_j = \frac{GR_i - \sum_j R_j}{G-1} = \frac{G}{G-1}\big(R_i - \mathrm{mean}_j\, R_j\big)$. Mean-centering is RLOO times the constant $(G-1)/G$: the direction is unbiased and only the scale changes. GRPO's real difference is the division by the standard deviation.

**Worked example | one success in four**

Rewards $(1, 0, 0, 0)$. Mean $0.25$; standard deviation $\sqrt{0.25 \cdot 0.75} = 0.433$. GRPO: $\hat{A} = (+1.73, -0.58, -0.58, -0.58)$. RLOO: $\hat{A} = (1-0,\ 0-\frac{1}{3},\ 0-\frac{1}{3},\ 0-\frac{1}{3}) = (+1, -0.33, -0.33, -0.33)$, which is exactly $\frac{4}{3} \times (0.75, -0.25, -0.25, -0.25)$.

**Pitfall.** Three traps hide in this formula.

- If all $G$ rewards are equal, every advantage is $0$ and the problem teaches nothing. DAPO's *dynamic sampling* drops such groups (Yu et al., 2025).
- With binary rewards $\mathrm{std} = \sqrt{p(1-p)}$, so dividing by it multiplies a problem's weight by $1/\sqrt{p(1-p)}$ compared with plain mean-centering. This up-weights problems that are almost always or almost never solved.
- The factor $1/|y_i|$ makes each token of a long wrong answer cost less, so the policy drifts toward longer wrong answers. Dr. GRPO removes both normalizations (Liu et al., 2025).

## 5 Trust regions: PPO clipping and the KL penalty

**Definition | clipped surrogate (Schulman et al., 2017)**

With the per-token ratio $\rho_t = \pi_\theta(y_t \mid x, y_{<t}) / \pi_{\text{old}}(y_t \mid x, y_{<t})$,

$$L^{\text{CLIP}} = \mathbb{E}\Big[\min\left(\rho_t \hat{A},\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\,\hat{A}\right)\Big].$$

**Worked example | what clipping does**

Take $\varepsilon = 0.2$. If $\hat{A} = +1$ and a token's probability already rose by $30\%$ ($\rho = 1.3$): $\min(1.3, 1.2) = 1.2$, a constant, so the gradient is zero and the update stops pushing that token. If $\hat{A} = -1$ and $\rho = 0.7$: $\min(-0.7, -0.8) = -0.8$, again constant. Clipping keeps each update inside a small trust region around $\pi_{\text{old}}$.

**The KL term.** GRPO estimates $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ per token with $k_3 = r - 1 - \log r$, where $r = \pi_{\text{ref}}/\pi_\theta$ and the token is sampled from $\pi_\theta$. Since $\mathbb{E}_{\pi_\theta}[r] = 1$, we get $\mathbb{E}[k_3] = \mathbb{E}[-\log r] = D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$: the estimate is unbiased and never negative.

**Why it matters for self-play.** R-Zero keeps a KL anchor ($\beta = 0.01$); Absolute Zero drops it. The anchor slows drift and collapse, but it also ties the policy to the base model, the very thing self-play hopes to go beyond (Section 6). Whether to anchor is a real design decision, not a detail.

### Exercises for Part I

**Exercise 1.** Compute the GRPO advantages for rewards $(1, 1, 1, 0)$. Which sample gets the largest weight, and why is that sensible?

**Exercise 2.** Show that GRPO with the standard deviation removed equals RLOO up to a constant factor. What is the factor for $G = 8$?

PART II

# What RL can and cannot do

## 6 The KL-regularized optimum: why RL mostly sharpens

**Key result | the tilted distribution**

$$\max_{\pi} \; \mathbb{E}_{y\sim\pi}[R(x,y)] - \beta\, D_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}}) \quad \Longrightarrow \quad \pi^\star(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\, e^{R(x,y)/\beta}}{Z(x)},$$

where $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x)\, e^{R(x,y)/\beta}$ makes the probabilities sum to one.

**Derivation (Gibbs variational identity).** Expand $D_{\mathrm{KL}}(\pi\|\pi^\star) = \sum_y \pi \log \frac{\pi}{\pi^\star} = D_{\mathrm{KL}}(\pi\|\pi_{\mathrm{ref}}) - \frac{1}{\beta}\mathbb{E}_\pi[R] + \log Z$. Multiplying by $-\beta$:

$$\mathbb{E}_\pi[R] - \beta D_{\mathrm{KL}}(\pi\|\pi_{\mathrm{ref}}) = \beta \log Z - \beta\, D_{\mathrm{KL}}(\pi\|\pi^\star).$$

The left side is our objective; the right side is largest when $D_{\mathrm{KL}}(\pi\|\pi^\star) = 0$, that is $\pi = \pi^\star$. The same identity gives DPO its closed form (Rafailov et al., 2023).

**Two consequences.** (i) *Support is kept:* if $\pi_{\mathrm{ref}}(y \mid x) = 0$, then $\pi^\star(y \mid x) = 0$. RL re-weights what the base model can already say. (ii) *Binary rewards multiply the odds:* with success probability $p$ under $\pi_{\mathrm{ref}}$,

$$\frac{p^\star}{1-p^\star} = \frac{p}{1-p}\, e^{1/\beta}.$$

**Worked example | amplification, and its limit**

With $\beta = 0.1$, $e^{1/\beta} = e^{10} \approx 22{,}026$.

| base success $p$ | $0.1$ | $10^{-3}$ | $10^{-6}$ |
|---|---|---|---|
| optimum $p^\star$ | $0.9996$ | $0.957$ | $0.022$ |

The optimum can turn a 1-in-1000 answer into a $96\%$ answer. But the optimum is not the training run: with $G$ samples per problem, the chance of seeing even one success is $1-(1-p)^G$. For $G = 8$ and $p = 10^{-3}$ this is $0.008$. In practice RL mostly amplifies what the base model already produces often.

**Why it matters for self-play.** This formula is the "sharpen versus expand" debate in one line. Self-play's real lever is different: the Challenger changes the *problems* $x$, building stepping stones on which $p(x)$ is not tiny. Re-weighting $y$ alone cannot do that.

## 7 pass@$k$, coverage and the sharpening signature

**Definition | pass@$k$ and its unbiased estimator (Chen et al., 2021)**

For one problem, $\text{pass@}k = 1 - (1-p)^k$. From $n \geq k$ samples with $c$ correct,

$$\widehat{\text{pass@}k} = 1 - \binom{n-c}{k} \Big/ \binom{n}{k}.$$

**Why:** pick $k$ of the $n$ samples at random; the pick fails only if all $k$ are wrong, which happens with probability $\binom{n-c}{k}/\binom{n}{k}$.

**Worked example | estimator**

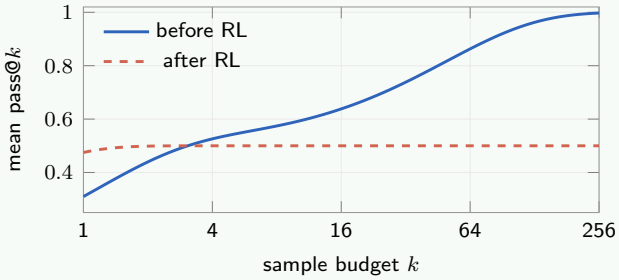
$n = 10$, $c = 2$, $k = 5$: $\binom{8}{5}/\binom{10}{5} = 56/252 = 0.222$, so pass@5 $= 0.778$. The plug-in value $1 - (1-0.2)^5 = 0.672$ is biased low on average, because $(1-\hat{p})^k$ is convex in $\hat{p}$ (Jensen's inequality).

**Worked example | the sharpening signature**

Two problems. Training moves A from $p = 0.60$ to $0.95$ and B from $p = 0.02$ to $0$.

| | before | after |
|---|---|---|
| pass@1 | 0.31 | 0.475 |
| pass@256 | 0.997 | 0.500 |

mean over problems A and B




For B before training, $1 - 0.98^{256} = 1 - e^{256 \ln 0.98} = 1 - e^{-5.17} = 0.994$. pass@1 rises while pass@256 falls: the model became more reliable on what it knew and lost what it could only sometimes do. The curves cross between $k = 3$ and $k = 4$. Yue et al. (2025) report this pattern for RLVR at large $k$.

**Why it matters for self-play.** Report pass@$k$ up to large $k$ (256 or more), and count *newly solved* problems: problems with base pass@256 $= 0$ that the trained model solves. Only those support the word "expand".

## 8 Entropy and exploration

The policy entropy $H(\pi(\cdot \mid x)) = -\sum_y \pi(y \mid x) \log \pi(y \mid x)$ measures how varied the samples are. As RL sharpens, entropy falls; groups become uniform; advantages become $0$; learning stops. This is *entropy collapse*. Common remedies are a larger upper clip range ("clip-higher", $\varepsilon_{\text{high}} > \varepsilon_{\text{low}}$, in DAPO), an entropy bonus, higher sampling temperature, and dropping zero-variance groups. Section 11 shows that a majority-vote reward is itself an entropy-minimizing objective.

### Exercises for Part II

**Exercise 3.** With $\beta = 0.2$ and base success $p = 0.01$, what is $p^\star$ at the KL-regularized optimum?

**Exercise 4.** Use the estimator with $n = 16$, $c = 1$, $k = 8$. Then show that with a single correct sample ($c = 1$) the estimate is always $k/n$.

PART III

# Learning signals: what should the Challenger make?

## 9 Learnability

**Key result | the $p(1-p)$ law**

For a binary reward with success probability $p$, $\mathrm{Var}[R] = p(1-p)$, largest $(1/4)$ at $p = 1/2$. A GRPO group of size $G$ carries a signal only if its rewards are not all equal:

$$\Pr[\text{informative group}] = 1 - p^G - (1-p)^G.$$

**Worked example | $G = 8$**

| $p$ (or $1-p$) | 0.5 | 0.9 | 0.99 |
|---|---|---|---|
| informative groups | 0.992 | 0.570 | 0.077 |

A problem solved $99\%$ of the time gives a signal in only $1$ group out of $13$. This is the arithmetic behind "make problems at the edge of the Solver's ability". R-Zero motivates the same target with a lower bound on the Solver's remaining learning potential that grows with $\hat{p}(1-\hat{p})$.

**Pitfall.** The law needs the *true* success rate. In label-free self-play we only measure agreement $\hat{p}$ with a pseudo-label, which equals $p$ only when the label is right.

## 10 Proposer rewards: a comparison

| Method | Challenger (proposer) reward | Truth from | Main limit |
|---|---|---|---|
| Asymmetric self-play (2017) | $r_A = \gamma \max(0,\, t_B - t_A)$, $r_B = -\gamma t_B$ (times to finish) | environment | needs a reversible world |
| PAIRED (2020) | regret $= U(\pi_{\text{antagonist}}) - U(\pi_{\text{protagonist}})$ | environment reward | antagonist must be strong |
| Absolute Zero (2025) | $1 - \bar{r}$ if $0 < \bar{r} < 1$, else $0$ ($\bar{r}$ = solve rate) | Python executor | code only |
| R-Zero (2025) | $\max\big(0,\ 1 - 2\lvert\hat{p} - \tfrac{1}{2}\rvert - \lambda\lvert C_k\rvert/B\big)$ | none (majority vote) | labels decay |
| SOAR (2026) | $\mathrm{Perf}_H(\mathrm{Train}(\theta, D_\phi)) - \mathrm{Perf}_H(\theta)$ | labeled hard set $H$ | needs labels on $H$ |
| Learning progress (2007) | decrease of the learner's error over time | environment | needs a measurable error |

**SOAR** (Sundaram, Kempe and Ollivier, ICML 2026) is the current accepted frontier. It is bilevel: the teacher writes a problem set $D_\phi$, the student trains on it briefly, and the teacher is paid by the student's real gain on hard problems it could not solve ($0$ of $128$ samples). Grounded rewards beat learnability rewards, and well-posed questions matter more than correct answers. Its gap is the labeled hard set $H$. **PowerPlay** (Schmidhuber, 2011) is the conceptual root: keep searching for the simplest problem the current solver cannot yet solve.

**Worked example | what R-Zero's reward cannot see**

Ten Solver answers: four say 20, four say 22, two have no boxed answer. Then $\hat{p} = 4/10$ and $r_{\text{unc}} = 1 - 2|0.4 - 0.5| = 0.8$, a high reward, and the question passes the 3–7 filter. In the released code the label is whichever of 20 and 22 appeared first (Python's `max` returns the first maximal key), and each unboxed answer counts as a vote for the string `"None"` (the question is dropped later only if `"None"` wins the vote). The reward cannot tell a two-way split from scattered guessing, and the label is a coin flip. Always read the code.

**Why it matters for self-play.** Rewards built only from the Solver's agreement measure difficulty, not truth. Rewards grounded in real progress (SOAR, PAIRED) need a source of truth. A grounded signal *without* labels is the open problem.

## 11 Learning from noisy labels

**Key result | random noise only shrinks the signal**

Suppose the observed reward $\tilde{R}$ flips the true reward with probability $\eta$, independently of which response was sampled. Then $\mathbb{E}[\tilde{R} \mid y] = \eta + (1 - 2\eta)R(y)$ and, because $\mathbb{E}[\nabla_\theta \log \pi_\theta] = 0$,

$$\mathbb{E}\big[\tilde{R}\, \nabla_\theta \log \pi_\theta\big] = (1 - 2\eta)\, \nabla_\theta J.$$

The gradient keeps its direction whenever $\eta < 1/2$; only its size shrinks.

**Key result | correlated noise biases the signal**

If the label error depends on the response, write $\tilde{R} = R + \delta(x, y)$. Then $\mathbb{E}[\tilde{R}\, \nabla_\theta \log \pi_\theta] = \nabla_\theta J + \mathbb{E}\big[\delta\, \nabla_\theta \log \pi_\theta\big]$: an extra push toward whatever responses $\delta$ favors.

**Worked example | the majority-vote reward**

The pseudo-label is the mode $m(x)$ of the Solver's own samples, and the reward is $\mathbf{1}[\text{answer}(y) = m(x)]$. Its expected value is $\pi_\theta(m \mid x)$, which is maximized by putting all mass on $m$, whether or not $m$ is correct. Example: $\pi(20) = 0.40$, $\pi(22) = 0.35$, other answers $0.25$, and the truth is 22. The vote makes 20 the label and training pushes $\pi(20)$ toward 1: the model becomes confidently wrong.

**Why it matters for self-play.** This is the mechanism behind "rise, then fall". Early on the mode is usually right (R-Zero's labels are $79\%$ correct in round 1), so sharpening helps. As the Challenger writes harder questions, the mode is wrong more often ($63\%$ correct by round 3), and the same sharpening now teaches errors.

### Exercises for Part III

**Exercise 5.** For $G = 4$ and $p = 0.75$, what fraction of groups is informative?

**Exercise 6.** With symmetric label noise $\eta = 0.3$, by what factor is the expected gradient scaled? What happens at $\eta = 0.5$?

PART IV

# Self-play as a game

## 12 Games, equilibria, and why self-play cycles

**Definition | zero-sum games and equilibria**

Player 1 gets payoff $u(a, b)$ and player 2 gets $-u(a, b)$. With mixed strategies $s, t$ and payoff matrix $U$, von Neumann's minimax theorem states $\max_s \min_t s^\top U t = \min_t \max_s s^\top U t$. A *Nash equilibrium* is a pair of strategies in which no player gains by changing alone.

**Key result | naive self-play can spiral**

On $f(u, v) = uv$, where $u$ minimizes and $v$ maximizes, simultaneous gradient steps $u_{t+1} = u_t - \eta v_t$, $v_{t+1} = v_t + \eta u_t$ give

$$u_{t+1}^2 + v_{t+1}^2 = (1 + \eta^2)\,(u_t^2 + v_t^2),$$

so the iterates spiral away from the equilibrium $(0, 0)$ for any step size.

In rock–paper–scissors, best responses cycle forever: rock, then paper, then scissors, then rock. The standard fixes:

- **Fictitious play:** best-respond to the *average* of all past opponents; the time-averages converge in two-player zero-sum games (Robinson, 1951).
- **Opponent pools:** play against a random earlier checkpoint, as AlphaGo did.
- **PSRO** (Lanctot et al., 2017): grow a population of policies and mix them by a meta-level Nash equilibrium.
- **Optimistic or extragradient updates:** these converge on bilinear games where plain gradient steps spiral.

**Why it matters for self-play.** Challenger–Solver self-play is general-sum and non-stationary: each side's target moves as the other learns. Forgetting earlier skills, the cross-round mode cycling reported by R-Diverse, is the LLM form of this cycling. Memory banks and populations are the LLM form of fictitious play.

## 13 Search as a teacher: expert iteration and AlphaZero

Expert iteration (Anthony et al., 2017) alternates two steps. An *improvement operator* $\mathcal{I}$, such as tree search, filtering or voting, turns the current policy into a stronger one; then the policy is trained to imitate it, $\pi \leftarrow \arg\min_\pi D_{\mathrm{KL}}(\mathcal{I}(\pi) \,\|\, \pi)$. AlphaZero is the famous case, with loss

$$\ell = (z - v)^2 - \boldsymbol{\pi}_{\mathrm{MCTS}}^\top \log \mathbf{p} + c\,\|\theta\|^2,$$

where $z$ is the real game result, $v$ the predicted value, $\boldsymbol{\pi}_{\mathrm{MCTS}}$ the search's move distribution and $\mathbf{p}$ the network's.

**Key result | the condition for self-improvement**

The loop improves only if $\mathcal{I}(\pi)$ is reliably better than $\pi$. In Go, search plus the true game result guarantees this. In label-free LLM self-play the operator is usually a majority vote, which is better than the policy only when the mode is right.

## 14 STaR as maximum marginal likelihood

**Key result | Fisher's identity**

With a latent rationale $z$ and gold answer $y^\star$,

$$\nabla_\theta \log p_\theta(y^\star \mid x) = \mathbb{E}_{z\sim p_\theta(z\mid x,y^\star)}\Big[\nabla_\theta \log p_\theta(z, y^\star \mid x)\Big].$$

**Derivation.** $\nabla_\theta \log p(y^\star \mid x) = \frac{1}{p(y^\star\mid x)} \sum_z \nabla_\theta p(z, y^\star \mid x) = \sum_z \frac{p(z,y^\star\mid x)}{p(y^\star\mid x)} \nabla_\theta \log p(z, y^\star \mid x)$, and the weight is the posterior $p(z \mid x, y^\star)$. If the answer is read off the rationale, $y = \mathrm{ans}(z)$, then sampling $z \sim p_\theta(z \mid x)$ and keeping it only when $\mathrm{ans}(z) = y^\star$ gives exact posterior samples. Fine-tuning on the kept rationales, as STaR does (Zelikman et al., 2022), is therefore a stochastic gradient of $\log p_\theta(y^\star \mid x)$: an EM-style algorithm.

**Key result | RL versus likelihood**

RL with a $0/1$ reward maximizes $p_\theta(y^\star \mid x)$, and $\nabla_\theta p = p\, \nabla_\theta \log p$. STaR maximizes $\log p_\theta(y^\star \mid x)$, and $\nabla_\theta \log p = \nabla_\theta p / p$. On a problem solved $1\%$ of the time, the likelihood objective weights the gradient $100$ times more than RL does: log-likelihood favors hard problems.

**Why it matters for self-play.** "Maximize $p$" versus "maximize $\log p$" decides whether rare successes on hard problems drive learning. It ties directly to the Challenger's job of keeping $p$ in the learnable range.

### Exercises for Part IV

**Exercise 7.** In the bilinear example with $\eta = 0.1$ and start $(u, v) = (1, 0)$, how far from the equilibrium are the iterates after $10$ steps?

**Exercise 8.** A problem has $p_\theta(y^\star \mid x) = 0.05$. By what factor does the likelihood gradient exceed the RL gradient on this problem?

PART V

# Where can new knowledge come from?

## 15 The data processing inequality

**Key result | information cannot be created by processing**

If $T \to X \to S$ is a Markov chain, meaning the signal $S$ is computed from $X$ alone, then

$$I(T;S) \le I(T;X).$$

Take $T$ to be the truth, $X$ the problem together with the model, and $S$ any label-free signal built from the model's own samples: votes, confidence, consistency checks. No such signal carries more information about the truth than the model already holds. Label-free self-play can *extract and concentrate* that information; it cannot *create* it. To expand, the loop needs an outside source (an executor, a proof checker, an environment) or an asymmetry such as checking being easier than solving.

**An important nuance.** The inequality bounds information, not usability. A model can "know" an answer at pass@256 and still fail at pass@1. Turning latent knowledge into reliable behavior is real value; it is sharpening, not new knowledge.

## 16 The generator–verifier gap

**Key result | filtering helps only if checking beats chance**

Filter self-generated answers with the model's own check, with $\mathrm{TPR} = \Pr(\text{accept} \mid \text{correct})$ and $\mathrm{FPR} = \Pr(\text{accept} \mid \text{wrong})$. By Bayes' rule, the accuracy among accepted answers is

$$\mathrm{prec} = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}}, \qquad \text{and} \qquad \mathrm{prec} > p \iff \mathrm{TPR} > \mathrm{FPR}.$$

**Proof.** $\mathrm{prec} > p \iff \mathrm{TPR} > p\,\mathrm{TPR} + (1-p)\mathrm{FPR} \iff (1-p)\mathrm{TPR} > (1-p)\mathrm{FPR}.$

**Worked example | a useful and a useless verifier**

$p = 0.3$, $\mathrm{TPR} = 0.9$, $\mathrm{FPR} = 0.2$: $\mathrm{prec} = 0.27/(0.27 + 0.14) = 0.659$, more than double the base accuracy, so training on accepted answers can help. If $\mathrm{TPR} = \mathrm{FPR}$, then $\mathrm{prec} = p$ and filtering adds nothing.

Song et al. (2025) formalize this generation–verification gap and study how it varies across models and verification methods. It is the cleanest statement of *when* self-improvement is possible at all.

**Why it matters for self-play.** A majority vote is a verifier too: its TPR and FPR come from how often right and wrong answers become the mode. When errors are shared, a wrong answer becomes the mode as easily as a right one, and the gap closes.

## 17 When does voting find the truth? The theory of shared errors

Majority voting is the default answer key of label-free self-play. This section asks exactly when the vote is right, why it fails, and what any replacement must do better. Together with Section 18, it is the most important part for finding a new method: it ends with a map of the channels a new signal can use (Section 17.8) and a worksheet for testing one (Section 17.9).

### 17.1 Independent voters: Condorcet's theorem and its rate

**Key result | Condorcet's jury theorem**

$n$ independent voters, each correct with probability $q$ on a two-option question:

$$P_{\text{maj}}(n) = \sum_{k>n/2} \binom{n}{k} q^k (1-q)^{n-k} \xrightarrow[n\to\infty]{} \begin{cases} 1, & q > 1/2 \\ 0, & q < 1/2. \end{cases}$$

**How fast.** For $q > 1/2$, Hoeffding's inequality gives $\Pr[\text{majority wrong}] \le \exp(-2n(q-\frac{1}{2})^2)$: the error vanishes exponentially.

**Worked example | independent voters**

$q = 0.6$, $n = 9$: the terms for $k = 5, \dots, 9$ are $0.251,\ 0.251,\ 0.161,\ 0.060,\ 0.010$, so $P_{\text{maj}} = 0.733$. Nine weak voters beat one. With $n = 101$ the bound gives an error of at most $0.133$; the exact error is $0.021$.

### 17.2 Many answers: the vote converges to the mode

**Key result | plurality voting**

Let $\pi(a \mid x)$ be the Solver's answer distribution and $t$ the true answer. As $n \to \infty$, the plurality vote returns the mode $\arg\max_a \pi(a \mid x)$ (when the mode is unique). For each wrong answer $w$ with $\pi(w \mid x) < \pi(t \mid x)$,

$$\Pr\left[\, w \text{ gets at least as many votes as } t \,\right] \le \exp\Big( -\tfrac{n}{2} \left(\pi(t \mid x) - \pi(w \mid x)\right)^2 \Big),$$

and a union bound over the wrong answers bounds the chance that the vote misses $t$.

**Derivation.** Score each sample $+1$ if it says $t$, $-1$ if it says $w$, and $0$ otherwise. The mean score is $\pi(t \mid x) - \pi(w \mid x)$ and each score lies in an interval of length $2$, so Hoeffding's inequality bounds the chance that the sum is $\le 0$ by $\exp(-2n\Delta^2/2^2) = \exp(-n\Delta^2/2)$.

**Two consequences.** (i) In the limit, the vote is right exactly when the truth is the mode; more samples only estimate the mode better. (ii) The *margin* $\Delta = \pi(t \mid x) - \max_{w \neq t} \pi(w \mid x)$ decides how many samples are needed.

**Worked example | a small margin is a coin flip**

The truth $22$ has $\pi = 0.40$, the wrong answer $20$ has $0.35$, and five other answers have $0.05$ each. With $10$ votes the plurality picks $22$ only $56\%$ of the time and $20$ in $42\%$ (exact multinomial count, ties split evenly). For this pair, the bound guarantees $90\%$ only after about $1{,}800$ votes. R-Zero's reward cannot see the danger: it reads only the top answer's share ($\hat{p} \approx 0.4$, reward $\approx 0.8$), not how close the runner-up is.

### 17.3 Three models of correlated errors

Across questions, votes are not independent. Three models describe how they depend on each other; for a single LLM the third one is exact.

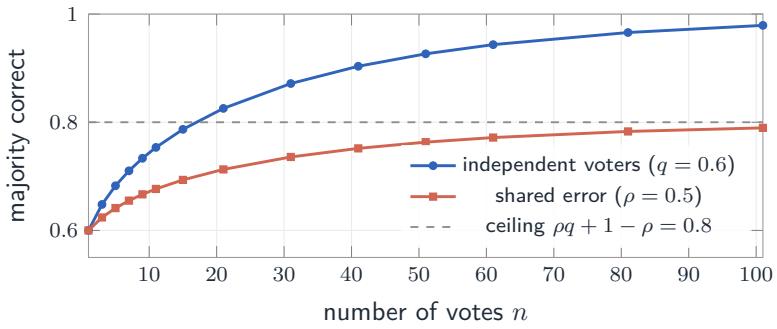
**Key result | (a) common shock: a shared error caps the vote**

With probability $\rho$ all voters copy one shared draw (correct with probability $q$); otherwise they vote independently. Then

$$P = \rho\, q + (1-\rho)\, P_{\text{maj}}(n) \xrightarrow[n\to\infty]{} \rho\, q + (1-\rho) < 1.$$

**Worked example | correlated voters**

$\rho = 0.5$, $q = 0.6$: $P = 0.3 + 0.5 \times 0.733 = 0.667$ at $n = 9$, and never more than $0.80$ however many samples are drawn. More samples cannot remove a shared error.




Independent votes climb toward 1; votes that share an error stop below the ceiling.

**Key result | (b) exchangeable voters: the effective number of votes**

If every pair of correctness indicators has correlation $r > 0$, then $\text{Var}(\text{mean}) = \frac{q(1-q)}{n}(1+(n-1)r)$, so the vote behaves like

$$n_{\text{eff}} = \frac{n}{1+(n-1)\,r} \xrightarrow[n\to\infty]{} \frac{1}{r}$$

independent votes, in the sense of the variance of the vote share (the survey-sampling *design effect*, Kish 1965; a rough guide, not an exact accuracy formula). With $r = 0.2$, ten samples are worth about $3.6$ independent ones, and a thousand are worth at most five.

**Key result | (c) latent difficulty: the ceiling of voting**

Samples are independent *given the question*, but each question $x$ has its own success rate $q(x)$. Then the vote's accuracy as $n \to \infty$ is

$$\Pr_x\left[q(x) > \tfrac{1}{2}\right] \text{ (two options)}, \qquad \text{or} \qquad \Pr_x\left[\text{the mode of } \pi(\cdot \mid x) \text{ is the truth}\right] \text{ (in general)}.$$

The share of questions whose mode is right is the hard ceiling of any vote.

**Worked example | voting lifts easy questions and sinks hard ones**

$60\%$ of questions have $q = 0.8$, $25\%$ have $q = 0.45$, and $15\%$ have $q = 0.1$. One sample is right $0.6 \cdot 0.8 + 0.25 \cdot 0.45 + 0.15 \cdot 0.1 = 60.75\%$ of the time; an infinite vote is right $60\%$ of the time. The vote lifts the easy questions to $100\%$ and drops the other $40\%$ to $0\%$, even though the model solves them sometimes.

**Key result | (c) explains (b)**

For one LLM sampled at a fixed prompt, model (c) is exact: samples are independent given the question, so all their correlation runs through the question itself. Averaged over questions, it produces exactly the correlation of model (b):

$$r = \frac{\mathrm{Var}_x[q(x)]}{\bar{q}\,(1-\bar{q})}, \qquad \bar{q} = \mathbb{E}_x[q(x)].$$

In the example above, $r = 0.067/0.238 = 0.28$.

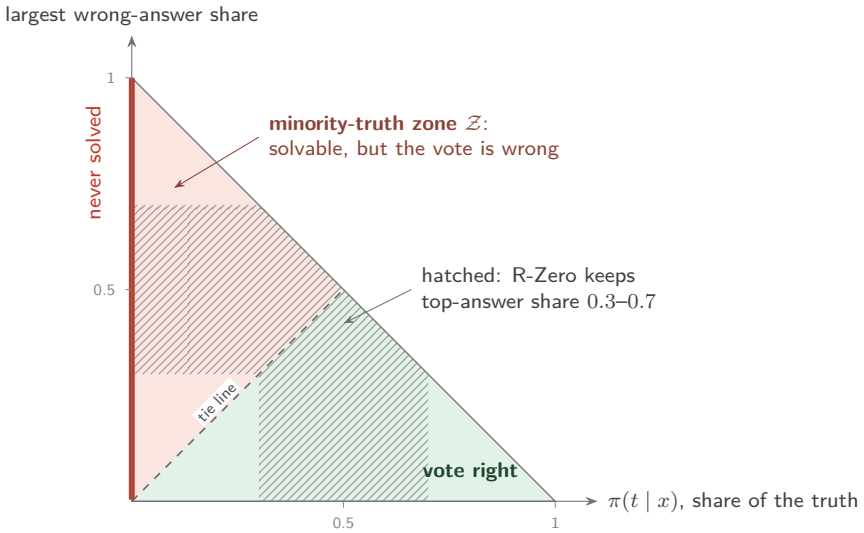
**Derivation.** For two samples $i \neq j$ of the same random question, $\mathrm{Cov}(X_i, X_j) = \mathbb{E}[q^2] - \bar{q}^2 = \mathrm{Var}_x[q]$ and $\mathrm{Var}(X_i) = \bar{q}(1-\bar{q})$.

So a "shared error" of one model is not noise between its samples: it is a *question whose mode is wrong*. Ensembles do not escape it either: Kim et al. (2025) find that larger and more accurate models make highly correlated errors, even across different providers.

**Definition | the minority-truth zone**

$$\mathcal{Z} = \Big\{ x : \; 0 < \pi(t \mid x) \leq \max_{w \neq t} \pi(w \mid x) \Big\}.$$

On $\mathcal{Z}$ the model can solve the problem, but the vote is wrong (or a coin flip, at a tie). A better label pays *only* here: outside $\mathcal{Z}$ the vote is already right, and on never-solved questions a correct label gives an outcome-reward learner no correct sample to reinforce.




Every question is a point. R-Zero's band straddles the tie line, so it can keep questions from $\mathcal{Z}$.

**Pitfall.** R-Zero keeps questions whose top answer has 30–70% of the votes, and its Challenger is paid most near 50%. That band crosses the tie line, so the kept set can contain many minority-truth questions, and a better Challenger can find more of them. This is consistent with R-Zero's label accuracy falling from 79% to 63% over three rounds.

### 17.4 The indistinguishability lemma

**Key result | a rule cannot separate what it cannot see**

Let a labeling rule $L$ read only an observation $O(x)$, for example the final answers of $n$ samples, their full reasoning traces, or hidden states, plus randomness independent of the truth. If two problems $x$ and $x'$ give $O(x)$ and $O(x')$ the same distribution, but their truths differ ($t \neq t'$), then

$$\Pr[L(x) = t] + \Pr[L(x') = t'] \leq 1.$$

Majority voting is the case where $O(x)$ is the final answers of samples from $\pi(\cdot \mid x)$.

**Proof.** $L(x)$ and $L(x')$ have the same distribution $D$ over labels, and $D(t) + D(t') \leq 1$ because $t \neq t'$.

**Corollary: the three escape routes.** A label that beats the vote must read something whose distribution *differs* between such a pair. There are three ways to read more:

1. **The model on other inputs** $\tilde{x}$: related problems, perturbations, inverse problems, or $x$ asked in a different way. This helps only if the model behaves differently on $\tilde{x}$ and $\tilde{x}'$.
2. **The model's internal states** $h(x)$: hidden activations, not just outputs. This helps only if $h(x)$ differs between the pair and carries the truth.
3. **Information from outside the model**: execution, an environment, or a program that reads the problem text itself. This differs by design.

**Worked example | why every consistency check passes a consistent misreading**

Suppose the model reads $x$ as a different problem $\hat{x}$ and solves $\hat{x}$ well. Then on $x$, and on every related input (other parameter values, paraphrases that keep the trigger, the inverse problem), it behaves exactly as it would on the correctly read $\hat{x}$-versions. Route 1 is closed: each consistency check verifies $\hat{x}$ perfectly, so votes, regularities across problem families and backward checks all agree with the wrong answer. Only something that compares the model's written setup with the original text, or an outside solver, can see the difference.

**Why it matters for self-play.** This is the single most useful test for a new idea. Before any experiment, write down exactly what your signal reads. Then write down two problems on which that looks the same but the truths differ, and ask whether your signal can tell them apart. If it cannot, no tuning will save it.

### 17.5 Where shared errors come from

| Source | Shared through | Can be exposed by | Cannot be exposed by |
|---|---|---|---|
| Misreading the problem | the meaning the model gives the text | a check of the written setup against the original text; an outside solver; internal states, if they encode the reading | votes; checks that reuse the same reading (families, inverse problems) |
| Wrong method or formula | the learned procedure | execution; a truly different method | votes; checks that use the same method |
| Missing knowledge | the pretraining data | outside sources (tools, retrieval) | anything inside the model |
| Consistent arithmetic slip | computation habits | executing the arithmetic | votes |
| Wording-induced bias | the surface form of the prompt | paraphrase, reordering | – |
| Answer-extraction artifact | the answer parser | better measurement; counting non-answers apart | – |

### 17.6 Classical fixes, and why one model makes them hard

**Dawid–Skene (1979).** Model each annotator $j$ by a confusion matrix and assume their errors are independent *given the truth*: $\Pr(\ell_1, \dots, \ell_m \mid t) = \prod_j \pi_j(\ell_j \mid t)$. EM then estimates the truth and every annotator's reliability. When annotators share an error, EM learns the shared error as the truth.

**Surprisingly popular (Prelec, Seung and McCoy, 2017).** Ask each voter for an answer *and* a prediction of how others will answer, then pick the answer that is more popular than predicted. Asked "Is Philadelphia the capital of Pennsylvania?", most people say yes (wrong); those who know the answer is Harrisburg also expect most people to say yes, so "no" turns out more popular than predicted, and wins. The Bayesian truth serum (Prelec, 2004) is a closely related scoring rule that pays answers for being surprisingly common.

**Pitfall.** Both fixes need voters whose errors are not all the same. Recent work adapts them to LLMs: aggregation across *several different* LLMs with second-order information (2510.01499), and the truth serum as a GRPO reward, used there against sycophancy (2608.25267). For a single model, all samples come from one mind, so its predictions about "other voters" may share its own error. Whether they carry independent information is an open question.

### 17.7 When does a new signal beat the vote?

**Key result | the override condition**

Let $e$ be the vote's error rate. A new signal $S$ overrides the vote when they disagree. Let $a = \Pr[S \text{ gives the truth} \mid \text{vote wrong}]$ (rescue rate) and $b = \Pr[S \text{ overrides with a wrong answer} \mid \text{vote right}]$ (break rate). Then

$$\Delta\text{acc} = e\,a - (1-e)\,b > 0 \iff \frac{a}{b} > \frac{1-e}{e}, \qquad \text{ER} = \frac{\Delta\text{acc}}{e} = a - b\,\frac{1-e}{e}.$$

**Derivation.** Overriding changes the outcome only where $S$ and the vote disagree: it gains on the wrong votes it rescues ($e\,a$) and loses on the right votes it breaks ($(1-e)\,b$).

**Worked example | the worse the vote, the easier it is to help**

| vote error $e$ | $0.21$ (R-Zero, round 1) | $0.37$ (round 3) | $0.50$ |
|---|---|---|---|
| needed ratio $a/b$ | $3.76$ | $1.70$ | $1.00$ |

A signal too weak to help in round 1 can help by round 3: new signals matter most where the vote is worst.

**Worked example | unsafe, and safe but weak**

Suppose the vote is wrong on $e = 0.3$ of the questions, so an override needs $a/b > 0.7/0.3 = 2.33$. A signal that rescues $a = 6\%$ of the wrong votes but breaks $b = 3.5\%$ of the right ones has $a/b = 1.71 < 2.33$, and accuracy falls: $\Delta\text{acc} = 0.3 \times 0.06 - 0.7 \times 0.035 = -0.0065$. A stricter version that switches only near-ties might rescue the same $a = 6\%$ and break only $b = 1.8\%$: now $a/b = 3.33$, above the threshold, yet it gains only $0.54$ points ($\text{ER} = 0.018$). Both numbers matter: the ratio for safety, and the rescue rate inside $\mathcal{Z}$ for size.

## 17.8 Channels of new information: a map

| Channel | Extra information | Beats a consistent misreading? |
|---|---|---|
| More samples, same model | none | no |
| Other checkpoints or seeds | little (same lineage) | rarely |
| Training dynamics | how fast an answer is learned | unclear |
| Perturbations, paraphrases, resampled numbers | the model on related inputs (route 1) | only if the error depends on surface form |
| Problem families and regularities | the model on related inputs (route 1) | no |
| Backward or inverse problems | the model on inverse inputs (route 1) | not if the reading is reused |
| Meta-predictions (surprisingly popular, truth serum) | beliefs about other voters | unclear for one model |
| Internal states (probes, latent knowledge) | hidden activations (route 2) | possibly: states can differ when outputs agree |
| Execution (code, proof checkers) | outside the model (route 3) | yes, for what can be executed |
| Other models (judges, ensembles) | another model | only if their errors differ |

Route 2 has real evidence behind it: models' internal representations often encode whether an answer is correct even when the output is wrong (Orgad et al., 2024), and contrast-consistent search finds truth-like directions without labels (Burns et al., 2022). Two cautions: most probes are trained with labels (contrast-consistent search is the label-free exception), and a Challenger rewarded against a probe may learn to fool it. Any probe should also be compared with the model's own token probabilities, a free baseline that often carries the same signal.

## 17.9 A worksheet for a new signal

1. **Name the shared cause** you want to beat (for example, a consistent misreading).
2. **Build the indistinguishable pair** on paper (Section 17.4): two problems on which everything your

signal reads looks the same, but the truths differ.

3. **Name your channel** (route 1, 2 or 3) and check that it differs on the pair. If not, stop.
4. **Check the ceiling**: on vote-wrong questions, count how often the truth appears among the samples at all. An answer key can fix at most that share; if it is low, the problem is finding the answer, not judging it (Section 18).
5. **Measure $a$ and $b$** on a small labeled pilot, separately on vote-wrong and vote-right questions. Require $a/b > (1-e)/e$ and a useful error reduction, decided before you look.
6. **Check it does not decay**: measure $a$ by difficulty bin and by round; it must hold where the vote is worst.
7. **Check the costs**: no labels, no judge model, cheap enough to run on every question in every round.
8. **Audit novelty** before any training run.

**Why it matters for self-play.** Six rules in one place. (1) More samples never fix a wrong mode. (2) New information pays only in the minority-truth zone. (3) A signal cannot separate two problems whose inputs to it look the same. (4) Consistency checks verify the wrong problem perfectly when the reading is wrong. (5) A useful signal needs both a high rescue-to-break ratio and a high rescue rate inside $\mathcal{Z}$. (6) No answer key can pick an answer that was never sampled: when the truth is missing, the fix is search, not a better judge.

## 18 Search and correctness: finding and recognizing rare answers

Section 17 showed that a vote cannot fix a question whose mode is wrong. A sharper fact makes this worse: when the vote is wrong, the right answer is often not among the samples at all. For GPT-3.5 on math word problems, about $40\%$ of the failures have no correct chain among the samples (FOBAR, Section 18.6), weaker models do worse, and the correct answers that do appear are often sampled at $1\%$ or lower (Section 18.3). So the next question has two halves, exactly the two things AlphaGo's search had and label-free self-play lacks: **find** the right answer, then **recognize** it without an answer key.

**Key result | one question, two steps**

Let $p$ be the chance that one sample is right, $k$ the number of samples, and let a check accept a right answer with probability $\mathrm{TPR}$ and a wrong one with probability $\mathrm{FPR}$. Then

$$\Pr[\text{found at least once}] = 1-(1-p)^k, \qquad k_{90\%} = \frac{\ln 10}{-\ln(1-p)} \approx \frac{2.3}{p}$$

samples are needed to find it with probability $0.9$, and an accepted sample is right with probability

$$\mathrm{prec} = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}} \qquad \text{(Section 16)}.$$

**Derivation.** The $k$ samples all miss with probability $(1-p)^k$; setting $(1-p)^k = 0.1$ and taking logs gives $k_{90\%}$, and $-\ln(1-p) \approx p$ for small $p$. The precision is Bayes' rule, as in Section 16.

**Worked example | why 10 samples are not enough**

A question the model solves $5\%$ of the time is found in 10 samples with probability $1-0.95^{10} = 0.40$. Finding it $90\%$ of the time needs $\ln 10/(-\ln 0.95) = 45$ samples; at $p = 1\%$ it needs $230$. A question with $p = 0$ is never found by sampling, at any $k$: only a different procedure (a hint, an easier stepping stone, a decomposition, new information) can reach it (Section 15).

### 18.1 The base-rate trap: rare answers need strict checks

**Key result | how strict the check must be**

Accepted answers are right more often than not, $\mathrm{prec} \geq \frac{1}{2}$, if and only if

$$\mathrm{FPR} \leq \frac{p}{1-p}\,\mathrm{TPR} \;\approx\; p\,\mathrm{TPR}.$$

The check's false-accept rate must be *smaller than the rate at which the right answer appears*.

**Derivation.** $\mathrm{prec} \geq \frac{1}{2} \iff p\,\mathrm{TPR} \geq (1-p)\,\mathrm{FPR}$.

**Worked example | a good-looking check that fails**

The right answer is rare, $p = 0.02$, and the check looks strong: $\mathrm{TPR} = 0.9$, $\mathrm{FPR} = 0.05$. Then $\mathrm{prec} = 0.018/(0.018 + 0.049) = 0.27$: almost three of every four "verified" answers are wrong. In counts, 100 samples give $1.8$ right answers accepted and $4.9$ wrong ones. With $\mathrm{FPR} = 0.005$ the precision is $0.018/(0.018 + 0.0049) = 0.79$. A false-accept rate about ten times smaller than the rate of the right answer makes search useful.

### 18.2 A verified vote and the shared-mistake condition

Accepting any checked sample wastes the vote's information. A better rule groups the accepted samples by answer and returns the answer with the most accepted samples (the plain vote if nothing is accepted). Let $\pi_a$ be the share of samples with answer $a$, let the truth $t$ pass the check with probability $\mathrm{TPR}$, and let each wrong answer $w$ pass with its own rate $\mathrm{FPR}_w$.

**Key result | when a verified vote finds the truth**

As $k \to \infty$ the verified vote returns the truth if and only if

$$\pi_t\,\mathrm{TPR} \;>\; \max_{w \neq t}\; \pi_w\,\mathrm{FPR}_w,$$

and for finite $k$, $\Pr[w \text{ ties or beats } t] \leq \exp\!\big(-\tfrac{k}{2}\,(\pi_t \mathrm{TPR} - \pi_w \mathrm{FPR}_w)^2\big)$.

**Derivation.** Answer $a$ collects about $k\,\pi_a\,\Pr[\text{accept} \mid a]$ accepted samples, so the law of large numbers picks the largest $\pi_a \Pr[\text{accept} \mid a]$. For the finite bound, score a sample $+1$ if it says $t$ and is accepted, $-1$ if it says $w$ and is accepted, $0$ otherwise; the mean is $\pi_t \mathrm{TPR} - \pi_w \mathrm{FPR}_w$, each score lies in an interval of length $2$, and Hoeffding's inequality gives the bound, exactly as for plurality voting in Section 17.

**Worked example | the check must reject the wrong mode, not wrong answers in general**

A shared mistake gives the wrong answer $w$ half the samples, $\pi_w = 0.5$, and the truth $\pi_t = 0.2$. With $\mathrm{TPR} = 0.9$ the verified vote finds the truth only if $\mathrm{FPR}_w < 0.2 \times 0.9/0.5 = 0.36$: the check must reject the *shared* wrong answer at least $64\%$ of the time. A check that rereads the problem the same wrong way accepts $w$ almost always ($\mathrm{FPR}_w \approx 1$), and the verified vote returns $w$ exactly like the plain vote. The average false-accept rate over all wrong answers can look excellent while $\mathrm{FPR}_w$ on the mode is the only number that matters.

**Pitfall.** Report a check's $\mathrm{TPR}$ and $\mathrm{FPR}$ *separately on the vote's wrong mode* (the questions where the vote is wrong), not averaged over all questions. Signals look better where the truth is already the majority, so a probe or confidence score can look strong overall and still fail inside the

minority-truth zone, the only place where it could help. Compare it there with the model's own token probability.

### 18.3 What more samples buy: the coverage law

Brown et al. (*Large Language Monkeys*) separate two quantities. **Coverage** is "the fraction of problems that are solved by any generated sample" (pass@$k$, Section 7); **precision** asks "how often can we identify correct samples from our collection of generations?"

**Definition | the coverage law (Brown et al., Eqs. 2–3)**

$$\log C \approx a\, k^b \qquad \Longleftrightarrow \qquad C \approx \exp(a\, k^b),$$

with $C$ the coverage, $k$ the number of samples and $a, b$ fitted ($b < 0$, so $C \to 1$ as $k \to \infty$). Inverting gives the budget for a target coverage: $k \approx (\ln C/a)^{1/b}$.

**Worked example | Llama-3-8B-Instruct on MATH**

The paper's fit is $a = -1.33$, $b = -0.43$. It predicts coverage $0.26$ at $k = 1$, $0.61$ at $k = 10$ and $0.83$ at $k = 100$ (the paper measures $82.9\%$ at 100 samples and $98.44\%$ at 10,000). Reaching $90\%$ coverage needs $k \approx (\ln 0.9/-1.33)^{-1/0.43} \approx 364$ samples.

**Key result | finding grows, choosing stalls**

On the same model and benchmark, coverage rises from $82.9\%$ to $98.44\%$ between 100 and 10,000 samples, while majority voting and reward-model selection rise only from $40.50\%$ to $41.41\%$. All selection methods "saturate before reaching 100 samples", because "the occurrence of rare, correct solutions does not affect the most common answer that majority voting chooses"; those correct solutions are sampled "at low probabilities (e.g. 1% or lower)".

With an automatic verifier, coverage becomes real accuracy: on SWE-bench Lite, DeepSeek-Coder-V2 rises from $15.9\%$ with one sample to $56\%$ with 250, above the single-sample state of the art ($43\%$). And rare correct answers are mostly real: over $90\%$ of the graded chains of thought were faithful, even where correct answers were rare. The needle is in the haystack; what is missing is a way to recognize it.

### 18.4 Why an imperfect check caps search

Stroebl, Kapoor and Narayanan (*The Limits of Inference Scaling Through Resampling*) study code with unit tests that sometimes pass wrong programs. They name the two error rates: **completeness**, the "probability of accepting a correct solution" (our $\mathrm{TPR}$), and **soundness**, the "probability of rejecting an incorrect solution" (our $1 - \mathrm{FPR}$). Below, $p$ is again the chance that one sample is correct (the paper calls it $r$).

**Key result | the resampling ceiling**

Resample until the check accepts. A false positive returned at any attempt is final, and "resampling cannot decrease this probability, so it imposes an upper bound to the accuracy of resampling-based inference scaling, regardless of compute budget". On one task type, the accuracy of the returned answer is the precision of Section 16,

$$\mathrm{prec} = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}},$$

whatever the budget $K$. Hence: if $P_{\text{strong}}(\text{correct}) > P_{\text{weak}}(\text{correct} \mid \text{pass})$, "the weaker model cannot match the performance of a single invocation of the stronger model, no matter how big the compute budget".

**Derivation.** Each attempt ends the search with a correct answer with probability $p\,\mathrm{TPR}$ and with a wrong one with probability $(1-p)\,\mathrm{FPR}$; otherwise it continues. Whenever the search ends, the answer is correct with probability $p\,\mathrm{TPR}/(p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR})$, so the accuracy does not depend on $K$; more attempts only raise the chance that the search ends at all.

**Worked example | easy and hard tasks**

The paper's two-type model for Llama-3.1-8B on HumanEval+ has easy tasks with $p = 0.87$, hard tasks with $p = 0.13$, $\mathrm{TPR} = 1$ and $\mathrm{FPR} = 0.25$. On hard tasks the precision is $0.13/(0.13 + 0.25 \times 0.87) = 0.37$: most accepted answers are wrong. Each rejection makes a hard task more likely: after $k-1$ rejections, $\Pr[\text{type } i] \propto \beta_i^{k-1} w_i$, where $w_i$ is the share of tasks of type $i$ and $\beta_i = (1-\mathrm{TPR})\,p_i + (1-\mathrm{FPR})(1-p_i)$ is its rejection rate. So the false-positive share *grows* with the number of attempts: "the easy tasks get solved within a few attempts, and for the remaining harder tasks, false positives are more likely". Measured: Llama-3.1-70B's false-positive share rises from about $0.13$ at $K = 1$ to about $0.65$ at large $K$, and when a false positive costs four times what a true one gains, the best budget is $K \le 5$ for all four models tested.

**Pitfall.** Their model treats rejections as independent draws. A self-check whose errors follow the model's own misreading is worse: it accepts the *same* wrong answer again and again. That is the case $\mathrm{FPR}_w \approx 1$ of the verified-vote condition, and no number of samples escapes it.

## 18.5 Where a strict check can come from

By the data processing inequality (Section 15), a check computed from the model's own outputs adds no new information. A check helps only through an *asymmetry*: verifying is easier than solving, or the check reads something the mistake does not control (Section 17.8). Four practical sources:

| Source | Why it is easier than solving | Where it breaks |
|---|---|---|
| Plug the answer back in | substituting 7 into $3x + 5 = 26$ is trivial | only for answers that can be substituted |
| Backward check (FOBAR) | recovering a masked number from the answer reads the problem in another direction | if the misreading is reused, the check confirms it |
| Answer-first construction | the answer is fixed before the question is written | the written text may not match the intended problem |
| Execution (Absolute Zero) | a program's output is computed, not judged | only for executable tasks (code) |

## 18.6 The backward check in practice: FOBAR

FOBAR (Jiang et al.) checks a candidate answer by turning the question around. Every number in the question is replaced, one at a time, by $x$, and the model is asked:

*"If we know the answer to the above question is $\{\hat{A}_j\}$, what is the value of unknown variable $x$?"*

**Definition | forward–backward scoring (FOBAR, Eqs. 1–4)**

For candidate answers $\hat{A}_j$ (the distinct forward answers, set $\mathcal{A}$), forward: $\mathbb{P}_F(\hat{A}_j) = \frac{1}{M_F}\sum_{i=1}^{M_F}\mathbf{1}(A_i = \hat{A}_j)$, the vote share. Backward: with $N_Q$ masked numbers and $M_B$ backward samples each,

$$Z_j = \sum_{n=1}^{N_Q}\sum_{b=1}^{M_B}\mathbf{1}(\widehat{\text{num}}_{j,b}^{(n)} = \text{num}^{(n)}), \qquad \mathbb{P}_B(\hat{A}_j) = \frac{Z_j + \epsilon}{\sum_{j'} Z_{j'} + \epsilon|\mathcal{A}|},$$

and the final answer maximizes $\mathbb{P}(\hat{A}_j) \propto \mathbb{P}_F(\hat{A}_j)^{\alpha}\,\mathbb{P}_B(\hat{A}_j)^{1-\alpha}$, with $\alpha = 0.5$ ($\alpha = 1$ is plain self-consistency). We write $j$ for the paper's candidate index $c$. The paper uses $M_B = 8$, temperature $0.7$, and exact string match.

**Connection.** With $\alpha = \frac{1}{2}$ the argmax of $\sqrt{\mathbb{P}_F\,\mathbb{P}_B}$ is the argmax of $\mathbb{P}_F\,\mathbb{P}_B \propto \pi_a\,\Pr[\text{check passes} \mid a]$: FOBAR is exactly the verified vote above, so its success condition is $\pi_t\,\text{TPR} > \pi_w\,\text{FPR}_w$.

**Worked example | what the paper's numbers say**

- **The check is weak.** A masked number is recovered $41$–$53\%$ of the time from the correct candidate, but also $9$–$25\%$ of the time from a wrong one (GPT-4: $25.0\%$ and $19.2\%$; Fig. 5). So $\text{TPR} \approx 0.5$ and $\text{FPR} \approx 0.1$–$0.25$ per backward sample.
- **Gains shrink as models get stronger.** Average over six datasets, self-consistency → FOBAR: $81.4 \to 83.5$ (text-davinci-003), $85.5 \to 86.6$ (GPT-3.5), $91.1 \to 91.4$ (GPT-4) (Table 1, CoT prompts).
- **The absent-answer ceiling is in the paper too.** For GPT-3.5, "about $60\%$ of the total failure cases have at least one correct chain (the remaining $40\%$ have no correct chains and thus cannot be solved by backward reasoning)"; FOBAR fixed $54$ of those $294$ (Sec. 4.8).
- **A shared misreading can pass the check.** In one example the gold answer is $1.9$ dollars, yet the backward score prefers $190$ (cents), $0.57$ to $0.43$ (App. C.2).

Plug the paper's rates into the verified-vote condition: with $\text{TPR} = 0.5$ and $\text{FPR}_w = 0.2$, the truth wins only if $\pi_t > 0.4\,\pi_w$; against a wrong mode with half the samples, the truth needs at least $20\%$ of them. The paper's own limitations: masking a *redundant* number makes it unpredictable, and choosing what to mask is hard outside math.

## 18.7 Self-play with a perfect check: Absolute Zero

Absolute Zero (Zhao et al.) is the cleanest AlphaGo-like loop for language models: one model proposes and solves tasks, and a Python executor is the referee. A task is a triplet $(p, i, o)$ with $o = p(i)$:

| Task | Given → predict | How the executor checks |
|---|---|---|
| Deduction | $(p, i) \to o$ | value equality of the predicted and true output |
| Abduction | $(p, o) \to i$ | $p(i_\pi) = o$; any input that gives the output counts, because programs need not be one-to-one |
| Induction | half of the $(i, o)$ pairs and a message $\to p$ | $p_\pi$ must reproduce *held-out* pairs, which blocks if-else memorization |

**Definition | rewards and advantages (Absolute Zero, Eqs. 4–8)**

Proposer: $r_{\text{propose}} = 0$ if $\bar{r}_{\text{solve}} = 0$, else $1 - \bar{r}_{\text{solve}}$, with $\bar{r}_{\text{solve}}$ the mean of $G = 8$ solver attempts. Solver: $r_{\text{solve}} = \mathbf{1}(y = y^\star)$ by value equality in Python. Format: $-0.5$ for a well-formatted wrong answer, $-1$ for a format error. Advantages use six separate baselines, one per (task, role):

$$A^{\text{norm}}_{\text{task,role}} = \frac{r - \mu_{\text{task,role}}}{\sigma_{\text{task,role}}}, \qquad \text{task} \in \{\text{ind}, \text{ded}, \text{abd}\},\ \text{role} \in \{\text{propose}, \text{solve}\}.$$

Proposed tasks are kept only if the program runs, uses no banned packages, and gives the same output twice (determinism check).

**Worked example | what transfers, and what does not**

AZR-Coder-7B raises the average over code and math benchmarks from $40.2$ to $50.4$, with math $+15.2$ points, and beats earlier zero-data models by $1.8$ points on average (Table 1). But its math score ($39.1$) stays below zero-RL models trained on curated math (for example $45.8$): the math gain is *transfer* from checked code, not checked math. Gains grow with size ($+5.7$, $+10.2$, $+13.2$ for 3B, 7B, 14B), and the authors report that composite programs collapsed to the trivial $f(g(x)) = g(x)$, a proposer gaming the format of its tasks. The lesson: with an executor, rare successes are recognized with $\text{FPR} \approx 0$; natural-language math has none, so the question is how close a cheap check can get to $\text{FPR}_w \approx 0$ on the shared wrong answer.

**Why it matters for self-play.** Together, these papers give one recipe for testing a new check. (1) **Find:** on the questions where the vote is wrong, measure how often the truth appears among the samples, and how that grows with $k$ (the coverage law). (2) **Recognize:** measure the check's $\text{TPR}$ on the truth and its $\text{FPR}_w$ on the vote's wrong answer, not on average, and test $\pi_t \,\text{TPR} > \pi_w \,\text{FPR}_w$ question by question. (3) **Same compute:** a check that costs as much as ten extra samples must beat a vote with ten extra samples. (4) **The ceiling:** if the check accepts the shared wrong answer about as often as the truth appears, more search makes the wrong answer more certain, not less.

## 19 Model collapse

**Key result | recursive training loses the tails**

Fit a Gaussian by maximum likelihood to $N$ samples drawn from the previous generation's fit. Since $\mathbb{E}[\hat{\sigma}^2] = \frac{N-1}{N}\sigma^2$ for the maximum-likelihood variance, iterating gives

$$\mathbb{E}[\sigma_t^2] = \Big(\frac{N-1}{N}\Big)^t \sigma_0^2 \ \longrightarrow\ 0.$$

**Worked example | how fast**

$N = 100$, $t = 100$: $(0.99)^{100} = 0.366$. Two thirds of the variance is gone, and the rare cases (the tails) go first.

Shumailov et al. (2024) show the same tail loss for language models trained recursively on their own outputs. Gerstgrasser et al. (2024) show that *accumulating* real and synthetic data, instead of replacing old data, keeps the error bounded.

**Why it matters for self-play.** R-Zero's Solver trains only on labels it made itself: the pure "replace" regime. Keeping earlier data, anchoring to the base model, and adding outside data are the known fixes.

## 20 Weak-to-strong generalization

**Definition | performance gap recovered (Burns et al., 2023)**

$$\mathrm{PGR} = \frac{\mathrm{Perf}_{\text{weak}\to\text{strong}} - \mathrm{Perf}_{\text{weak}}}{\mathrm{Perf}_{\text{strong ceiling}} - \mathrm{Perf}_{\text{weak}}}.$$

Strong models trained on a weaker model's labels often recover a substantial part of the gap to their ceiling: labels worse than the student can still teach it. This is the optimistic counterpart of model collapse.

**Why it matters for self-play.** Noisy self-labels can still teach, as long as their errors are not the ones the student already believes. Self-labels are risky because their errors are, by construction, the student's own beliefs.

### Exercises for Part V

**Exercise 9.** Condorcet with $n = 3$ and $q = 0.55$: what is $P_{\mathrm{maj}}$?

**Exercise 10.** A verifier has $\mathrm{TPR} = 0.8$ and $\mathrm{FPR} = 0.4$ on a problem set with $p = 0.2$. What is the accuracy of the accepted answers?

**Exercise 11.** In the Gaussian collapse model with $N = 50$, what fraction of the variance remains after $50$ generations?

**Exercise 12.** Samples have pairwise error correlation $r = 0.1$. How many independent votes are $50$ samples worth, and what is the limit for many samples?

**Exercise 13.** The vote is wrong on $30\%$ of questions. A new signal rescues $20\%$ of wrong votes and breaks $5\%$ of right ones. Does overriding help? Compute $\Delta$acc and ER.

**Exercise 14.** Half of the questions have $q = 0.9$ and half have $q = 0.3$. Compute the single-sample accuracy, the accuracy of an infinite vote, and the between-sample error correlation $r$.

**Exercise 15.** A question is solved by $3\%$ of samples. How many samples give a $90\%$ chance of finding the right answer at least once?

**Exercise 16.** A check has $\mathrm{TPR} = 0.8$ and $\mathrm{FPR} = 0.1$, and the right answer appears in $5\%$ of samples. What share of accepted answers is right? What is the largest $\mathrm{FPR}$ that keeps it at or above one half?

**Exercise 17.** The truth has $\pi_t = 0.3$ of the samples and the vote's wrong answer has $\pi_w = 0.4$. The check accepts the truth with $\mathrm{TPR} = 0.9$. Below what $\mathrm{FPR}_w$ on the wrong answer does a verified vote pick the truth for large $k$?

PART VI

# Evidence reviewers trust

## 21 How big is the noise?

**Key result | benchmark uncertainty**

On $N$ problems with accuracy near $p$, the standard error is $\sqrt{p(1-p)/N}$ and a $95\%$ interval is about $\pm 1.96$ standard errors.

| benchmark | problems $N$ | standard error at $p = 0.5$ | $95\%$ half-width |
|---|---|---|---|
| AIME (one year) | $30$ | $0.091$ | $\pm 17.9$ points |
| MATH-500 | $500$ | $0.022$ | $\pm 4.4$ points |
| GSM8K test | $1{,}319$ | $0.014$ | $\pm 2.7$ points |

Averaging $k$ samples per problem (avg@$k$) removes sampling noise, not the noise from which problems happen to be in the set. Compare two methods with a *paired* bootstrap over the same problems.

## 22 A checklist before you claim anything

- **Same compute:** match rollouts, tokens and optimizer steps against every baseline.
- **Seeds:** at least three for training curves; a "rise, then fall" claim needs the fall to exceed the seed spread.
- **pass@$k$:** the unbiased estimator with $n \gg k$, reported up to large $k$.
- **Newness:** report base pass@256 and count problems the base model never solves.
- **Contamination:** exact and $13$-gram overlap between generated problems and every benchmark.
- **Decision rules first:** write the pass/fail rule before the run, and never pick checkpoints on test sets.
- **Read the code:** published numbers come from code, not from equations (Section 10).

PART VII

# Open problems: where a new thread can lead

1. **Growth.** Pure self-play peaks after about three to five rounds, and later Challengers do not teach more than earlier ones (*What Actually Evolves in Self-Evolving RL?*, ICLR 2027 submission). Collapse fixes such as R-Diverse and data gating (Survive or Collapse) stop the fall, but not the stall. *Goal:* a loop that keeps improving.
2. **Answer keys under shared errors.** A label-free signal that beats the vote on the minority-truth zone (Section 17), using a channel that a shared misreading does not control (Sections 17.4 and 17.8).
3. **Search with a strict check.** Find rare right answers and recognize them without labels. The check's false-accept rate on the vote's wrong answer must stay below the truth's share of the samples (Section 18).
4. **Measured expansion.** Newly solved problems at large $k$, beyond the base model's pass@256 (Section 7), instead of pass@1 gains that may be sharpening.
5. **Grounded teacher rewards without labels.** SOAR shows that rewarding real student progress works, but it needs a labeled hard set (Section 10).
6. **A theory of closed-loop improvement.** When can a loop with no outside information improve, and by how much? The data processing inequality (Section 15) and the verifier gap (Section 16) mark the boundary.

## A four-week study plan

| Week | Read in this guide, then derive by hand | Read the papers |
|---|---|---|
| 1 | Part I: policy gradient, baselines, the RLOO identity, PPO clipping | Sutton and Barto, ch. 13 [2]; PPO [3]; GRPO [4]; RLOO [5] |
| 2 | Part II: the tilted optimum, pass@$k$ and its estimator | DPO derivation [8]; pass@$k$ [9]; Yue et al. [10]; Dr. GRPO [7] |
| 3 | Part III: learnability, proposer rewards, noisy labels | Absolute Zero [15]; R-Zero [16]; SOAR [17]; PAIRED [12] |
| 4 | Parts IV–V: games, STaR, information limits, voting, search + check, collapse | STaR [24]; Mind the Gap [28]; Large Language Monkeys [43]; resampling limits [44]; FOBAR [45]; model collapse [29, 30]; weak-to-strong [31] |

**The test of understanding.** For each key result, you should be able to (1) derive it on a blank page, (2) redo its worked example without notes, and (3) say in one sentence what it implies for a Challenger–Solver loop.

## Appendix A Formula sheet

| Idea | Formula | Section |
|---|---|---|
| Policy gradient | $\nabla_\theta J = \mathbb{E}[R\,\nabla_\theta \log \pi_\theta(y \mid x)]$ | 2 |
| Baseline | $\mathbb{E}[b(x)\nabla_\theta \log \pi_\theta] = 0, \quad \hat{A} = R - b$ | 3 |
| GRPO | $\hat{A}_i = (R_i - \text{mean}\, R)/(\text{std}\, R + \epsilon)$ | 4 |
| RLOO | $\hat{A}_i = R_i - \frac{1}{G-1}\sum_{j\neq i} R_j = \frac{G}{G-1}(R_i - \text{mean}\, R)$ | 4 |
| PPO clip | $\min\big(\rho\hat{A},\ \text{clip}(\rho, 1\pm\varepsilon)\hat{A}\big), \quad \rho = \pi_\theta/\pi_{\text{old}}$ | 5 |
| KL estimator | $k_3 = r - 1 - \log r, \quad r = \pi_{\text{ref}}/\pi_\theta$ | 5 |
| KL optimum | $\pi^\star \propto \pi_{\text{ref}}\, e^{R/\beta}; \quad \text{odds}^\star = \text{odds} \cdot e^{1/\beta}$ | 6 |
| pass@$k$ | $1-(1-p)^k; \quad 1 - \binom{n-c}{k}/\binom{n}{k}$ | 7 |
| Learnability | $\text{Var}\, R = p(1-p); \quad$ informative group $1 - p^G - (1-p)^G$ | 9 |
| R-Zero reward | $\max(0,\ 1 - 2\lvert\hat{p} - \frac{1}{2}\rvert - \lambda\lvert C_k\rvert/B)$ | 10 |
| Absolute Zero | $1-\bar{r}$ if $0 < \bar{r} < 1$, else $0$ | 10 |
| SOAR | $\text{Perf}_H(\text{Train}(\theta, D_\phi)) - \text{Perf}_H(\theta)$ | 10 |
| PAIRED regret | $U(\pi_{\text{ant}}) - U(\pi_{\text{pro}})$ | 10 |
| Symmetric noise | $\mathbb{E}[\tilde{R}\,\nabla_\theta \log \pi] = (1-2\eta)\nabla_\theta J$ | 11 |
| Bilinear spiral | $u^2 + v^2$ grows by $(1+\eta^2)$ per step | 12 |
| Fisher identity | $\nabla_\theta \log p(y^\star \mid x) = \mathbb{E}_{p(z\mid x, y^\star)}[\nabla_\theta \log p(z, y^\star \mid x)]$ | 14 |
| RL vs likelihood | $\nabla_\theta p = p\,\nabla_\theta \log p$ | 14 |
| Data processing | $I(T;S) \le I(T;X)$ for $T \to X \to S$ | 15 |
| Verifier gap | $\text{prec} = \frac{p\text{TPR}}{p\text{TPR} + (1-p)\text{FPR}}; \quad \text{prec} > p \iff \text{TPR} > \text{FPR}$ | 16 |
| Condorcet | $\sum_{k>n/2}\binom{n}{k} q^k (1-q)^{n-k}; \quad$ error $\le e^{-2n(q-1/2)^2}$ | 17.1 |
| Plurality margin | $\Pr[w \text{ ties or beats } t] \le e^{-n(\pi_t - \pi_w)^2/2}$ | 17.2 |
| Shared-error cap | $\rho q + (1-\rho)P_{\text{maj}}(n) \to \rho q + 1 - \rho$ | 17.3 |
| Effective votes | $n_{\text{eff}} = n/(1 + (n-1)r) \to 1/r$ | 17.3 |
| Voting ceiling | $\Pr_x[\text{mode of } \pi(\cdot \mid x) \text{ is the truth}]$ | 17.3 |
| Indistinguishability | $O(x) \overset{d}{=} O(x'),\ t \neq t' \Rightarrow \Pr[L(x){=}t] + \Pr[L(x'){=}t'] \le 1$ | 17.4 |
| Correlation from difficulty | $r = \text{Var}_x[q(x)]/\big(\bar{q}(1-\bar{q})\big)$ | 17.3 |
| Override gain | $\Delta\text{acc} = ea - (1-e)b > 0 \iff a/b > (1-e)/e$ | 17.7 |
| Samples to find | $1-(1-p)^k; \quad k_{90\%} = \ln 10/(-\ln(1-p)) \approx 2.3/p$ | 18 |
| Strict check | $\text{prec} \ge \frac{1}{2} \iff \text{FPR} \le \frac{p}{1-p}\text{TPR}$ | 18.1 |
| Verified vote | truth wins $\iff \pi_t\,\text{TPR} > \max_w \pi_w\,\text{FPR}_w$ | 18.2 |
| Coverage law | $C \approx \exp(a\,k^b), \ \ b < 0$ | 18.3 |
| Resampling ceiling | accuracy $\le$ prec (Section 16), for any budget $K$ | 18.4 |
| FOBAR | $\mathbb{P} \propto \mathbb{P}_F^{\alpha}\,\mathbb{P}_B^{1-\alpha}, \ \ \alpha = \frac{1}{2}$ | 18.6 |
| Model collapse | $\mathbb{E}[\sigma_t^2] = ((N-1)/N)^t \sigma_0^2$ | 19 |
| Weak-to-strong | $\text{PGR} = \frac{\text{w2s}-\text{weak}}{\text{ceiling}-\text{weak}}$ | 20 |
| Standard error | $\sqrt{p(1-p)/N}$ | 21 |

## Appendix B Answers to the exercises

1. Mean $0.75$, standard deviation $0.433$: the correct samples get $+0.577$ and the wrong one $-1.732$. On an easy problem the rare failure is the most informative sample.
2. Without the division, $\hat{A}_i = R_i - \operatorname{mean} R = \frac{G-1}{G} \times \mathrm{RLOO}_i$. For $G = 8$ the factor is $7/8$.
3. Odds $0.0101 \times e^5 = 0.0101 \times 148.4 = 1.50$, so $p^\star = 1.50/2.50 = 0.60$.
4. $1 - \binom{15}{8}/\binom{16}{8} = 1 - 6435/12870 = 0.5$. In general, $\binom{n-1}{k}/\binom{n}{k} = (n-k)/n$, so the estimate is $k/n$.
5. $1 - 0.75^4 - 0.25^4 = 1 - 0.316 - 0.004 = 0.680$.
6. $1 - 2\eta = 0.4$. At $\eta = 0.5$ the factor is $0$: the labels carry no signal.
7. The squared radius grows by $1.01$ per step: $\sqrt{1.01^{10}} = 1.051$.
8. $1/p = 20$.
9. $q^3 + 3q^2(1-q) = 0.166 + 0.408 = 0.575$.
10. $0.16/(0.16 + 0.32) = 0.333$, above the base $0.2$, so filtering helps.
11. $(49/50)^{50} = e^{50 \ln 0.98} = 0.364$.
12. $n_{\mathrm{eff}} = 50/(1 + 49 \times 0.1) = 8.5$, and the limit is $1/r = 10$.
13. Needed ratio $0.7/0.3 = 2.33$; here $a/b = 4$, so yes. $\Delta\mathrm{acc} = 0.3 \times 0.2 - 0.7 \times 0.05 = 0.025$ (2.5 points); $\mathrm{ER} = 0.2 - 0.05 \times 2.33 = 0.083$.
14. Single sample $\bar{q} = 0.6$; infinite vote $0.5$ (only the easy half). $\mathrm{Var}[q] = 0.45 - 0.36 = 0.09$, so $r = 0.09/0.24 = 0.375$.
15. $k = \ln 10/(-\ln 0.97) = 2.303/0.0305 = 75.6$, so $76$ samples (the rule of thumb $2.3/p$ gives 77).
16. $\mathrm{prec} = 0.04/(0.04 + 0.095) = 0.30$. It reaches one half when $\mathrm{FPR} \le \frac{p}{1-p}\mathrm{TPR} = \frac{0.05}{0.95} \times 0.8 = 0.042$.
17. $\mathrm{FPR}_w < \pi_t \,\mathrm{TPR}/\pi_w = 0.27/0.4 = 0.675$: the check must reject the wrong answer at least about a third of the time.

## References


[1] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 1992.
[2] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
[3] J. Schulman et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
[4] Z. Shao et al. DeepSeekMath (introduces GRPO). arXiv:2402.03300, 2024.
[5] A. Ahmadian et al. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs (RLOO). arXiv:2402.14740, 2024.
[6] Q. Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476, 2025.
[7] Z. Liu et al. Understanding R1-Zero-Like Training: A Critical Perspective (Dr. GRPO). arXiv:2503.20783, 2025.
[8] R. Rafailov et al. Direct Preference Optimization. arXiv:2305.18290, 2023.
[9] M. Chen et al. Evaluating Large Language Models Trained on Code (unbiased pass@$k$). arXiv:2107.03374, 2021.
[10] Y. Yue et al. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837, 2025.
[11] S. Sukhbaatar et al. Intrinsic Motivation and Automatic Curricula via Asymmetric Self-Play. arXiv:1703.05407, 2017.
[12] M. Dennis et al. Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design (PAIRED). arXiv:2012.02096, 2020.
[13] J. Schmidhuber. PowerPlay: Training an Increasingly General Problem Solver by Continually Searching for the Simplest Still Unsolvable Problem. arXiv:1112.5309, 2011.
[14] P.-Y. Oudeyer, F. Kaplan and V. Hafner. Intrinsic motivation systems for autonomous mental development. *IEEE Trans. Evolutionary Computation*, 2007.
[15] A. Zhao et al. Absolute Zero: Reinforced Self-play Reasoning with Zero Data. arXiv:2505.03335, 2025.
[16] C. Huang et al. R-Zero: Self-Evolving Reasoning LLM from Zero Data. arXiv:2508.05004, 2025.
[17] S. Sundaram, J. Kempe and Y. Ollivier. Teaching Models to Teach Themselves: Reasoning at the Edge of Learnability (SOAR). arXiv:2601.18778, ICML 2026.
[18] R-Diverse: Mitigating Diversity Illusion in Self-Play LLM Training. arXiv:2602.13103, 2026.

[19] Survive or Collapse: The Asymmetric Roles of Data Gating and Reward Grounding in Self-Play RL. arXiv:2605.22217, 2026.
[20] J. Robinson. An iterative method of solving a game. *Annals of Mathematics*, 1951.
[21] M. Lanctot et al. A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning (PSRO). arXiv:1711.00832, 2017.
[22] T. Anthony, Z. Tian and D. Barber. Thinking Fast and Slow with Deep Learning and Tree Search (expert iteration). arXiv:1705.08439, 2017.
[23] D. Silver et al. Mastering the game of Go without human knowledge. *Nature*, 2017; and Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm, arXiv:1712.01815.
[24] E. Zelikman et al. STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465, 2022.
[25] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171, 2022.
[26] Y. Zuo et al. TTRL: Test-Time Reinforcement Learning. arXiv:2504.16084, 2025.
[27] T. M. Cover and J. A. Thomas. *Elements of Information Theory*, 2nd ed. Wiley, 2006.
[28] Y. Song et al. Mind the Gap: Examining the Self-Improvement Capabilities of Large Language Models. arXiv:2412.02674, ICLR 2025.
[29] I. Shumailov et al. AI models collapse when trained on recursively generated data. *Nature*, 2024 (arXiv:2305.17493).
[30] M. Gerstgrasser et al. Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. arXiv:2404.01413, 2024.
[31] C. Burns et al. Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision. arXiv:2312.09390, 2023.
[32] D. Silver and R. S. Sutton. Welcome to the Era of Experience. 2025.
[33] W. Hoeffding. Probability inequalities for sums of bounded random variables. *J. American Statistical Association*, 1963.
[34] L. Kish. *Survey Sampling*. Wiley, 1965 (design effect).
[35] A. P. Dawid and A. M. Skene. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Applied Statistics*, 1979.
[36] D. Prelec. A Bayesian truth serum for subjective data. *Science*, 2004.
[37] D. Prelec, H. S. Seung and J. McCoy. A solution to the single-question crowd wisdom problem. *Nature*, 2017.
[38] E. Kim, A. Garg, K. Peng and N. Garg. Correlated Errors in Large Language Models. arXiv:2506.07962, ICML 2025.
[39] C. Burns, H. Ye, D. Klein and J. Steinhardt. Discovering Latent Knowledge in Language Models Without Supervision. arXiv:2212.03827, 2022.
[40] H. Orgad et al. LLMs Know More Than They Show: On the Intrinsic Representation of LLM Hallucinations. arXiv:2410.02707, 2024.
[41] Beyond Majority Voting: LLM Aggregation by Leveraging Higher-Order Information. arXiv:2510.01499, 2025.
[42] Mitigating LLM sycophancy with RL-based fine-tuning: Bayesian Truth Serum approach. arXiv:2608.25267, 2026.
[43] B. Brown et al. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787, 2024.
[44] B. Stroebl, S. Kapoor and A. Narayanan. The Limits of Inference Scaling Through Resampling. arXiv:2411.17501, 2024.
[45] W. Jiang et al. Forward-Backward Reasoning in Large Language Models for Mathematical Verification (FOBAR). arXiv:2308.07758, 2023.